1.EL OBJETIVO DE LA SESIÓN 006

El objetivo de esta guía era dejar de guardar los datos solo en archivos y pasarlos a una base de datos de verdad. Teníamos que crear una base de datos en MySQL con sus tablas relacionadas, y después practicar los comandos SQL para insertar, consultar, actualizar y relacionar la información del GameStore Escolar, cuidando que no se rompa la integridad referencial.

2. QUÉ HERRAMIENTAS USÉ

Esta vez ya no usé VS Code para Java, sino que usé MySQL. Instalé MySQL Community Server y para manejarlo usé MySQL Workbench, que es el que recomendaba la guía porque ahí se ven las tablas y los resultados de las consultas más fácil. Si no tienes Workbench, también se puede con DBeaver o con la consola de MySQL, es lo mismo.



3. CÓMO ARMÉ LA ESTRUCTURA

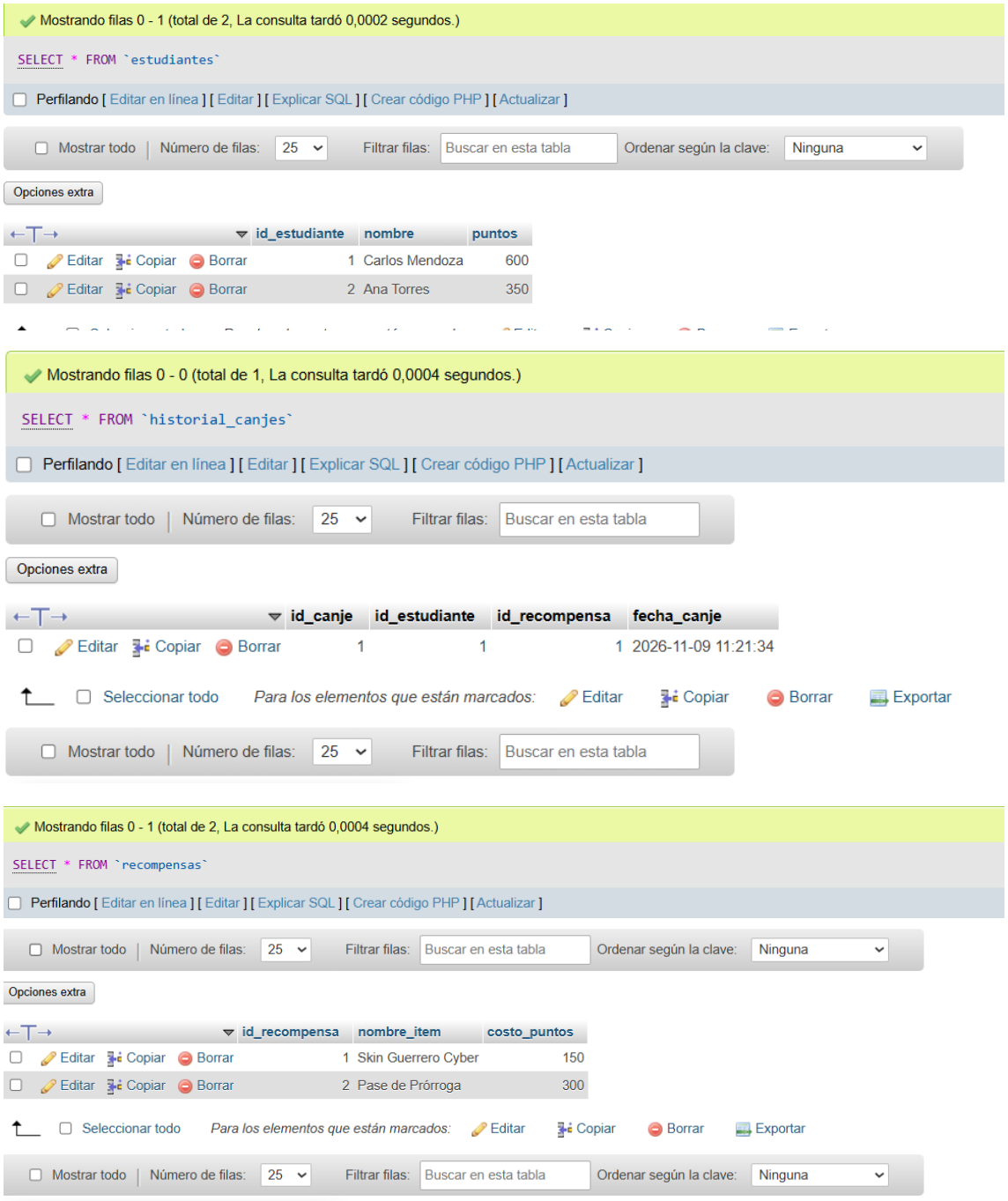
Primero creé una base de datos que se llama gamestore_db. Dentro de esa base de datos creé tres tablas que están relacionadas entre sí:

- estudiantes: guarda el id, el nombre y los puntos del estudiante.

- recompensas: guarda el id, el nombre del ítem y cuánto cuesta en puntos.

- historial_canjes: guarda el id del canje, el id del estudiante, el id de la recompensa y la fecha del canje.

La idea es que historial_canjes no guarda los nombres repetidos, solo guarda los ids. Así si un estudiante cambia de nombre, no tengo que cambiarlo en todos lados, solo en estudiantes. Eso lo entendí con el ejemplo del archivador que decía la guía: cada tabla es una carpeta y la llave foránea es la nota que dice ver la ficha de la otra carpeta.



4. CÓMO DESARROLLÉ CADA PASO

Trabajo 1 - Crear la base de datos y las tablas

Este fue el primer paso. Primero puse CREATE DATABASE IF NOT EXISTS gamestore_db para crear la base de datos si es que no existía, y después USE gamestore_db para entrar a trabajar dentro de ella.

Después creé la tabla estudiantes. Le puse id_estudiante como INT AUTO_INCREMENT PRIMARY KEY. El AUTO_INCREMENT es para que el id

se ponga solo, no tenga que ponerlo yo a mano, y el PRIMARY KEY es para que ese id sea único y no se repita nunca. El nombre lo puse VARCHAR(100) NOT NULL porque es texto y es obligatorio, y puntos lo puse INT DEFAULT 0 para que si no le pongo puntos, empiece en 0.

Luego creé la tabla recompensas igual: id_recompensa como PRIMARY KEY con AUTO_INCREMENT, nombre_item VARCHAR(100) NOT NULL y costo_puntos INT NOT NULL.

Al final creé la tabla historial_canjes, que es la más importante porque es la que une a las otras dos. Le puse id_canje como PRIMARY KEY, id_estudiante e id_recompensa como INT normales, y fecha_canje como TIMESTAMP DEFAULT CURRENT_TIMESTAMP para que la fecha se ponga sola con la fecha y hora del momento del canje.

Ahí mismo le puse las FOREIGN KEY. Una que dice que id_estudiante hace referencia a estudiantes(id_estudiante) y otra que id_recompensa hace referencia a recompensas(id_recompensa). Eso significa que no puedo poner un canje con un estudiante que no exista, porque MySQL me lo va a rechazar. Eso es la integridad referencial. Lo dejé explicado con un comentario dentro del script como pedía la guía.



TRABAJO 2 - INSERTAR DATOS Y HACER CONSULTAS

En el segundo paso ya empecé a meter datos de prueba con INSERT INTO.

Primero inserté dos estudiantes: Carlos Mendoza con 500 puntos y Ana Torres con 350. Después inserté dos recompensas: Skin Guerrero Cyber que cuesta 150 puntos y Pase de Prórroga que cuesta 300.

Después hice un SELECT _ FROM estudiantes para ver si sí se habían guardado. Ahí me salieron los dos con sus puntos originales, el de Carlos todavía en 500.

Después hice un UPDATE para cambiarle los puntos a Carlos a 600. Lo hice con WHERE id_estudiante = 1, porque si no le pongo WHERE me cambia los puntos de todos los estudiantes, y eso es el error que decía la guía que no hay que cometer. Por eso siempre revisé dos veces el WHERE antes de darle ejecutar.

Luego registré un canje con INSERT INTO historial_canjes poniendo solo los ids: id_estudiante 1 e id_recompensa 1. O sea que Carlos canjeó el Skin Guerrero Cyber. No puse la fecha porque se pone sola.

Al final hice la consulta más difícil, la del JOIN. Esa junta las tres tablas en una sola consulta para que se vea bonito: el id del canje, el nombre del estudiante, el nombre del ítem, cuánto costó y la fecha. Para eso usé JOIN estudiantes ON y JOIN recompensas ON, que son las condiciones para que MySQL sepa cómo unir las tablas por los ids.





5. CÓMO LO EJECUTÉ Y QUÉ ME SALIÓ

Para ejecutarlo fui seleccionando cada parte del script en Workbench y dándole al rayito de ejecutar. La primera vez me daba miedo que me salga error en las llaves foráneas, pero no, porque primero creé estudiantes y recompensas y después historial_canjes, y también primero inserté los estudiantes y después el canje. Si lo hacía al revés, sí me hubiera dado error.

El resultado final fue el que pedía la guía: en el primer SELECT Carlos salía con 500, y en el JOIN ya salía su canje con el Skin y su fecha. La fecha me salió diferente a la de la guía porque es la fecha real de cuando yo lo ejecuté, y eso está bien porque es CURRENT_TIMESTAMP.